

SHAP-анализ:

интерпретация моделей машинного обучения

Методическое пособие с практическими примерами

Подготовил Бессонов А.Р

Содержание

1. Введение: зачем интерпретировать модели
 2. Теоретические основы: от вектора Шепли к SHAP
 - 2.1. Игровая постановка и вектор Шепли
 - 2.2. Аксиомы Шепли
 - 2.3. Адаптация к машинному обучению
 3. SHAP: определение, свойства и формула
 4. Типы SHAP-объяснителей
 - 4.1. TreeExplainer
 - 4.2. KernelExplainer
 - 4.3. DeepExplainer и LinearExplainer
 - 4.4. Сравнительная таблица
 5. Визуализации SHAP: как читать графики
 - 5.1. Summary Plot (пчелиный улей)
 - 5.2. Bar Plot
 - 5.3. Waterfall Plot
 - 5.4. Dependence Plot
 - 5.5. Force Plot
 - 5.6. Decision Plot
 6. Сравнение с другими методами интерпретации
 7. Практические примеры применения
 - 7.1. Медицинская диагностика
 - 7.2. Финансовый скоринг
 - 7.3. Отбор признаков
 - 7.4. Поиск утечек данных и отладка модели
 8. Ограничения метода
 9. Заключение
 10. Список литературы
- Приложение: Jupyter Notebook с примерами

1. Введение: зачем интерпретировать модели

Современные модели машинного обучения — градиентный бустинг, случайный лес, нейронные сети — достигают впечатляющей точности в задачах классификации, регрессии и прогнозирования. Однако за эту точность приходится платить: чем сложнее модель, тем труднее понять, почему она приняла то или иное решение.

Представьте: банк отказывает клиенту в кредите. На вопрос «почему?» сотрудник отвечает: «Так решила модель». Это неубедительно. А если врач получает диагноз от нейросети и не может объяснить пациенту, на основании каких симптомов сделан вывод? Это уже не просто неудобство — это потенциальная опасность.

Проблема интерпретации моделей машинного обучения — или, как её часто называют, проблема «чёрного ящика» — стала одной из центральных в современном data science. Регуляторы (GDPR в Европе, законодательство США) всё чаще требуют, чтобы автоматизированные решения можно было объяснить человеку.

Что такое SHAP?

SHAP (SHapley Additive exPlanations) — это метод объяснения предсказаний моделей машинного обучения, основанный на теоретико-игровом подходе. Он был предложен в 2017 году Скоттом Лундбергом (Scott Lundberg) и Су-Ин Ли (Su-In Lee) и быстро стал одним из самых популярных инструментов интерпретации.

В основе SHAP лежит вектор Шепли (Shapley vector) — концепция из кооперативной теории игр, названная в честь нобелевского лауреата Ллойда Шепли. Идея проста и элегантна: если представить, что признаки объекта — это игроки в коалиции, а предсказание модели — это выигрыш коалиции, то «справедливая доля» каждого признака в этом выигрыше и будет его SHAP-значением.

SHAP решает три ключевые задачи интерпретации:

- Глобальная интерпретация: какие признаки в целом важны для модели?
- Локальная интерпретация: почему модель выдала именно такое предсказание для конкретного объекта?
- Анализ взаимодействий: как признаки влияют друг на друга?

В этом пособии мы последовательно разберём теоретические основы SHAP, его математические свойства, основные типы объяснителей и способы визуализации. Практическая часть оформлена в виде Jupyter Notebook — вы сможете запустить код, поэкспериментировать с параметрами и увидеть все графики своими глазами.

2. Теоретические основы: от вектора Шепли к SHAP

2.1. Игровая постановка и вектор Шепли

Представьте, что несколько человек объединяются для совместной работы. Вместе они создают некоторую ценность (выигрыш). Как справедливо разделить этот выигрыш между участниками? Именно этот вопрос решает вектор Шепли.

Формально: пусть есть множество игроков $N = 1, 2, \dots, n$ и характеристическая функция $v(S)$, которая для любой коалиции $S \subseteq N$ возвращает её выигрыш. Вектор Шепли $\varphi_i(v)$ определяет долю каждого игрока i по формуле:

$$\varphi_i(v) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|! (n - |S| - 1)!}{n!} \cdot [v(S \cup \{i\}) - v(S)]$$

$$S \subseteq N \setminus \{i\}$$

Смысл формулы: для каждого возможного подмножества S (не включающего i) мы смотрим, насколько выигрыш увеличивается при добавлении игрока i к коалиции S .

Эта разница $v(S \cup \{i\}) - v(S)$ называется предельным вкладом игрока i . Затем мы усредняем эти предельные вклады по всем возможным порядкам вступления игроков в коалицию.

Ключевая идея: справедливое распределение должно учитывать вклад игрока во всевозможные комбинации с другими участниками, а не только его индивидуальную производительность.

2.2. Аксиомы Шепли

Вектор Шепли — единственное решение, удовлетворяющее четырём естественным аксиомам:

- **Эффективность (Efficiency):** сумма долей всех игроков равна общему выигрышу.
 $\sum_i \varphi_i(v) = v(N)$. То есть ничего не теряется и не появляется из ниоткуда.
- **Симметричность (Symmetry):** если два игрока вносят одинаковый вклад в любую коалицию, то их доли равны. Справедливость в чистом виде.
- **Аксиома болвана (Dummy):** если игрок не добавляет ценности ни к одной коалиции, его доля равна нулю. Бесполезные игроки не получают ничего.
- **Аддитивность (Additivity):** если игра состоит из двух независимых частей, то доля игрока равна сумме его долей в каждой части. $\varphi_i(v + w) = \varphi_i(v) + \varphi_i(w)$.

Именно аксиоматическая обоснованность делает вектор Шепли особенно привлекательным: это не просто «какой-то» способ оценки важности, а математически единственно верный способ, удовлетворяющий интуитивным требованиям справедливости.

2.3. Адаптация к машинному обучению

SHAP перекладывает теорию игр на почву машинного обучения следующим образом:

- Игроки = признаки объекта (features).
- Коалиция = подмножество признаков, значения которых известны.
- Выигрыш коалиции $v(S)$ = условное математическое ожидание предсказания модели при известных признаках из S : $E[f(x)|x_S]$.
- Доля игрока ϕ_i = вклад признака i в предсказание модели для конкретного объекта.

Таким образом, SHAP-значение признака показывает, насколько изменилось предсказание модели по сравнению со средним (базовым) предсказанием, благодаря тому, что мы знаем значение этого конкретного признака.

Например, если модель предсказывает цену дома, и средняя цена по всем домам составляет \$200,000 (базовое значение), а для конкретного дома модель выдала \$350,000, то SHAP-значения покажут, какие именно признаки и на сколько «подняли» цену выше среднего. Скажем, большой метраж добавил +\$80,000, престижный район +\$50,000, а старый год постройки убавил -\$20,000.

3. SHAP: определение, свойства и формула

SHAP (SHapley Additive exPlanations) объединяет вектор Шепли с аддитивным представлением объяснения. Модель объяснения g для объекта x строится как:

$$g(z') = \varphi_0 + \sum \varphi_i \cdot z'_i$$

где $z' \in 0,1^M$ — бинарный вектор наличия признаков (1 — признак присутствует, 0 — отсутствует), M — количество признаков, φ_0 — базовое значение (среднее по всем объектам), а φ_i

— SHAP-значение для признака i .

Ключевое свойство SHAP — аддитивность. Это означает, что предсказание модели $f(x)$ в точности равно сумме базового значения и всех SHAP-значений:

$$f(x) = \varphi_0 + \varphi_1 + \varphi_2 + \dots + \varphi_M$$

Это невероятно удобно: вы всегда можете проверить, что сумма вкладов признаков плюс базовое значение действительно даёт предсказание модели. Никакой «магии» — чистая математика.

Три свойства SHAP

- Локальная точность (Local accuracy): предсказание модели для конкретного объекта в точности равно сумме SHAP-значений плюс базовое значение. $f(x) = \varphi_0 + \sum \varphi_i$.
- Отсутствие влияния (Missingness): если признак отсутствует (не использовался моделью), его SHAP-значение равно нулю. $z'_i = 0 \Rightarrow \varphi_i = 0$.
- Согласованность (Consistency): если модель изменилась так, что вклад признака увеличился (или остался прежним), его SHAP-значение не может уменьшиться.

Эти три свойства выделяют SHAP среди других методов интерпретации. Многие популярные методы, такие как LIME или встроенная важность признаков в деревьях решений, не удовлетворяют одновременно всем трём условиям. SHAP — единственный метод, который это гарантирует.

Почему SHAP, а не просто Feature Importance?

Стандартная важность признаков (feature importance) показывает, насколько модель «полагается» на каждый признак в среднем. Но она не отвечает на вопросы:

- В каком направлении влияет признак? Увеличивает он предсказание или уменьшает?
- Одинаково ли влияние для всех объектов или есть нюансы?
- Как признаки взаимодействуют друг с другом?

SHAP отвечает на все эти вопросы. Более того, в отличие от Permutation Importance, SHAP не требует многократного переобучения модели. А в отличие от LIME, он даёт математически согласованные и непротиворечивые объяснения, а не локальные аппроксимации.

4. Типы SHAP-объяснителей

Одно из главных преимуществ SHAP — наличие специализированных вычислителей (explainers) для разных типов моделей. Каждый эксплейнер оптимизирован под свою архитектуру модели и обеспечивает наилучший баланс скорости и точности.

4.1. TreeExplainer

TreeExplainer предназначен для древовидных моделей: Random Forest, XGBoost, LightGBM, CatBoost, Decision Tree и любых других моделей на основе деревьев решений.

Это самый быстрый и точный из всех эксплейнеров SHAP. Он использует внутреннюю структуру деревьев для вычисления SHAP-значений без какой-либо аппроксимации — результат получается точным (exact), а не приближённым. Для леса из 100 деревьев на 1000 объектов TreeExplainer выдаёт результат за секунды, тогда как KernelExplainer потребовал бы часы.

Ключевые возможности TreeExplainer:

- Вычисление точных SHAP-значений (не приближений)
- Анализ взаимодействий признаков через `shap_interaction_values`
- Поддержка многоклассовой классификации
- Специализированные визуализации для древовидных моделей

4.2. KernelExplainer

KernelExplainer — универсальный инструмент, работающий с любой моделью, которая реализует `predict()`. Он основан на методе Kernel SHAP — аппроксимации вектора Шепли через решение взвешенной задачи линейной регрессии.

Принцип работы: KernelExplainer генерирует «возмущённые» версии исходного объекта (случайно «включая» и «выключая» признаки), вызывает модель для каждой версии и строит локальную линейную модель, веса которой интерпретируются как SHAP-значения.

Плюсы:

- Работает с любой моделью
- Не требует доступа к внутренностям модели

Минусы:

- Медленный — время растёт экспоненциально с количеством признаков
- Результат — аппроксимация, а не точное значение
- Требуется фоновый датасет (background dataset) для маргинализации

4.3. DeepExplainer и LinearExplainer

DeepExplainer (Deep SHAP) оптимизирован для нейронных сетей, построенных с использованием TensorFlow/Keras или PyTorch. Он объединяет идеи из DeepLIFT и SHAP, обеспечивая быстрое вычисление SHAP-значений для глубоких моделей.

LinearExplainer работает с линейными моделями (линейная/логистическая регрессия) и вычисляет SHAP-значения аналитически, используя коэффициенты модели и корреляции признаков.

4.4. Сравнительная таблица

Эксплейнер	Типы моделей	Точность	Скорость	Взаимодействия
TreeExplainer	Древья, XGBoost, LightGBM, CatBoost	Точная	★★★	Да
KernelExplainer	Любые модели	Приближённая	★★☆	Нет
DeepExplainer	Нейронные сети (TF, PyTorch)	Приближённая	★★☆	Нет
LinearExplainer	Линейные модели	Точная	★★★	Нет

Практическая рекомендация: если ваша модель — дерево решений или ансамбль деревьев, всегда используйте TreeExplainer. Это быстро, точно и даёт максимум информации. KernelExplainer приберегите для моделей, с которыми другие эксплейнеры не работают (например, SVM или собственные реализации).

5. Визуализации SHAP: как читать графики

Одна из причин популярности SHAP — богатый арсенал визуализаций. Каждый тип графика решает свою задачу: от глобального анализа важности признаков до объяснения одного-единственного предсказания. Рассмотрим основные.

5.1. Summary Plot («пчелиный улей»)

Summary Plot — визитная карточка SHAP. Он показывает одновременно три вещи:

- Важность признака — чем выше признак в списке, тем он важнее.
- Направление влияния — точки справа от нуля увеличивают предсказание, слева — уменьшают.
- Значение признака — цвет точки: красный = высокое значение, синий = низкое.

Каждая точка на графике — это один объект из выборки. «Пчелиный улей» формируется группировкой точек по вертикали, чтобы избежать наложения.

Как читать Summary Plot:

- Если для признака красные точки справа, а синие слева — признак имеет монотонную положительную связь с предсказанием.
- Если точки образуют «бабочку» (расходятся в обе стороны при одном значении признака) — налицо нелинейность или взаимодействие признаков.
- Если точки узкой вертикальной полосой — влияние признака линейно и однозначно.

5.2. Bar Plot

Упрощённая версия Summary Plot. Показывает среднее абсолютное SHAP-значение для каждого признака — то есть его среднюю «силу влияния» на предсказание. Хорош для быстрой оценки: какие признаки модель считает главными?

Недостаток: не показывает направление влияния и разброс SHAP-значений. Признак с малым средним модулем может для отдельных объектов иметь огромное значение — но bar plot этого не отразит.

5.3. Waterfall Plot

Waterfall Plot — это объяснение одного конкретного предсказания. Он показывает, как каждый признак «сдвинул» предсказание от базового значения $E[f(x)]$ до конечного $f(x)$. Красные столбцы — признаки, увеличившие предсказание. Синие — уменьшившие.

Это идеальный инструмент для ответа на вопрос «почему модель приняла такое решение?». В медицинской диагностике, кредитном скоринге, HR-аналитике — везде, где нужно объяснить решение по конкретному случаю, Waterfall Plot незаменим.

5.4. Dependence Plot

Показывает зависимость SHAP-значения от значения самого признака. Каждая точка — объект. Позволяет увидеть:

- Нелинейные зависимости (SHAP-значение не растёт монотонно с признаком).
- Пороговые эффекты (после определённого значения влияние резко меняется).
- Взаимодействия (при раскрашивании вторым признаком видно, как один признак модулирует влияние другого).

Dependence Plot особенно полезен для проверки гипотез. Например, вы ожидаете, что «возраст» должен увеличивать риск заболевания после 60 лет. График покажет, действительно ли модель это «выучила».

5.5. Force Plot

Компактная визуализация, показывающая «борьбу сил»: красные силы толкают предсказание вверх, синие — вниз. Для одного объекта получается одна «полоска», для нескольких — интерактивный stacked force plot, позволяющий увидеть кластеры похожих объяснений.

5.6. Decision Plot

Показывает траекторию накопления предсказания: начиная с базового значения, каждый признак добавляет или вычитает свой вклад, пока не получится итоговое предсказание. Особенно нагляден при наложении траекторий нескольких объектов: сразу видно, у каких объектов решение формируется похожим образом.

Сводная таблица визуализаций

График	Уровень анализа	Показывает	Лучше всего для
Summary Plot (beeswarm)	Глобальный	Важность + направление + распределение	Первый взгляд на модель
Bar Plot	Глобальный	Средняя важность	Быстрая оценка, отчёт руководству
Waterfall Plot	Локальный	Вклад признаков в одно предсказание	Объяснение конкретного решения
Dependence Plot	Признак	Нелинейность, пороги, взаимодействия	Проверка гипотез, исследование признаков
Force Plot	Локальный	Компактное объяснение одного/нескольких объектов	Презентации, интерактивные отчёты
Decision Plot	Популяционный	Траектория накопления вклада	Сравнение объектов, поиск паттернов

6. Сравнение с другими методами интерпретации

SHAP не единственный метод интерпретации моделей.

Рассмотрим альтернативы и сравним их с SHAP.

Встроенная важность признаков (Feature Importance)

Многие модели (деревья, XGBoost) имеют встроенный механизм оценки важности признаков — обычно на основе того, насколько часто признак используется для разбиений и насколько эти разбиения уменьшают ошибку.

- Плюс: быстро, всегда доступно.
- Минус: смещено в пользу признаков с большим количеством уникальных значений (категориальные признаки с высокой кардинальностью). Не показывает направление влияния. Не учитывает взаимодействия.

Permutation Importance

Признак перемешивается (переставляется случайным образом), и измеряется падение качества модели. Чем сильнее упало качество, тем важнее признак.

- Плюс: не зависит от модели, интуитивно понятен.
- Минус: требует многократного прогона модели. При коррелированных признаках даёт смещённые оценки. Не показывает направление и не даёт локальных объяснений.

LIME (Local Interpretable Model-agnostic Explanations)

LIME строит локальную линейную аппроксимацию вокруг конкретного объекта и объясняет предсказание через коэффициенты этой линейной модели.

- Плюс: работает с любой моделью, быстро.
- Минус: объяснения могут быть нестабильными (разные при разных запусках). Локальная аппроксимация не гарантирует глобальной согласованности.

Partial Dependence Plots (PDP)

PDP показывает, как в среднем меняется предсказание при изменении одного или двух признаков.

- Плюс: наглядно, показывает форму зависимости.
- Минус: усреднение может скрывать важные эффекты. Предполагает независимость признаков. Не работает для отдельных объектов.

Сравнительная таблица методов

Метод	Глобальный	Локальный	Направление	Взаимодействия	Теор. обоснование
SHAP	Да	Да	Да	Да	Вектор Шепли (теория игр)
Feature Importance	Да	Нет	Нет	Нет	Эвристика
Permutation Imp.	Да	Нет	Нет	Нет	Эвристика
LIME	Нет	Да	Да	Нет	Локальная аппроксимация
PDP	Да	Нет	Да	Частично	Визуальный анализ

Как видно из таблицы, SHAP — единственный метод, закрывающий все аспекты интерпретации одновременно. Именно это делает его стандартом де-факто в индустрии.

7. Практические примеры применения

7.1. Медицинская диагностика

Контекст: модель предсказывает вероятность злокачественной опухоли по 30 признакам (размер, текстура, форма клеток и т.д.). Точность — 97%, но врачам нужно понимать, на каких основаниях сделан вывод.

Что даёт SHAP:

- Summary Plot показывает, что ключевые признаки — worst concave points и worst perimeter. Врач может проверить, согласуются ли они с медицинской практикой.
- Waterfall Plot для конкретного пациента: видно, что «хорошие» признаки (гладкость, симметричность) тянут в сторону «доброкачественная», но «плохие» (вогнутость, периметр) перевешивают.
- Dependence Plot подтверждает пороговый эффект: риск резко возрастает при worst concave points > 0.15.

Практический пример реализован в прилагаемом Jupyter Notebook — раздел «Задача классификации: Breast Cancer Wisconsin».

7.2. Финансовый скоринг

Контекст: банк использует градиентный бустинг для оценки кредитоспособности. Регулятор требует объяснять причины отказа.

Что даёт SHAP:

- Для каждого заявителя формируется Waterfall Plot с причинами решения. Это удовлетворяет требованиям GDPR о «праве на объяснение».
- Глобальный анализ выявляет дискриминационные паттерны: если признак «пол» или «возраст» имеет большое SHAP-значение, это повод для проверки модели на этическую предвзятость.
- Сравнение SHAP-значений у одобренных и отклонённых заявок показывает, какие факторы являются решающими.

7.3. Отбор признаков

Контекст: у вас 500 признаков, и нужно оставить 20 самых информативных.

Что даёт SHAP:

- Bar Plot ранжирует признаки по средней важности — верхние N можно оставить, остальные отбросить.
- В отличие от корреляционного анализа, SHAP учитывает нелинейные зависимости и взаимодействия.
- После сокращения набора признаков можно переобучить модель и сравнить SHAP-графики — важность оставшихся признаков должна сохранить структуру.

SHAP-отбор признаков особенно эффективен, когда важны не просто корреляции, а именно влияние на целевую переменную в контексте модели.

7.4. Поиск утечек данных и отладка модели

SHAP помогает находить проблемы в данных и модели:

- Если признак имеет подозрительно высокую важность — возможно, это утечка данных (например, признак, недоступный в момент предсказания).
- Если важный с точки зрения предметной области признак имеет $SHAP \approx 0$ — модель его игнорирует, возможно, ошибка в предобработке.
- Dependence Plot может вскрыть аномалии: например, модель «считает», что с ростом дохода риск дефолта растёт, — это противоречит здравому смыслу и указывает на ошибку в данных или переобучение.

8. Ограничения метода

При всех достоинствах SHAP не является серебряной пулей. Важно знать о его ограничениях:

Вычислительная сложность

Вычисление точных SHAP-значений требует перебора 2^M подмножеств признаков, что экспоненциально по M . На практике это решается использованием специализированных эксплейнеров (TreeExplainer для деревьев, DeepExplainer для нейросетей) или аппроксимаций (KernelExplainer с ограниченным числом сэмплов). Но для «чёрного ящика» с сотнями признаков KernelExplainer может работать часы.

Коррелированные признаки

При сильной корреляции признаков SHAP-значения могут быть нестабильными. Метод предполагает независимость признаков при маргинализации, и когда два признака несут похожую информацию, SHAP может «размазать» важность между ними или приписать её одному случайным образом. Это не ошибка метода, а фундаментальное ограничение, связанное с невозможностью однозначно разделить вклад коррелированных признаков.

Причинно-следственная связь

SHAP объясняет корреляционные связи, а не причинно-следственные. То, что признак имеет большое SHAP-значение, не означает, что изменение этого признака приведёт к изменению целевой переменной. Для причинного анализа нужны другие методы (Causal Inference, Do-calculus).

Зависимость от фонового датасета

KernelExplainer и некоторые другие эксплейнеры требуют фонового датасета (background) для вычисления условных математических ожиданий. Результат зависит от выбора этого датасета. Если фоновая выборка нерепрезентативна, SHAP-значения могут быть смещены.

9. Заключение

SHAP — это мощный, теоретически обоснованный и практически удобный инструмент интерпретации моделей машинного обучения. Его ключевые преимущества:

- Единственный метод интерпретации с аксиоматическим обоснованием (вектор Шепли).
- Работает на трёх уровнях: глобальном, локальном и популяционном.
- Богатый набор визуализаций для разных задач: от быстрой оценки до детального объяснения.
- Специализированные эксплейнеры для разных типов моделей (деревья, нейросети, линейные модели, любые чёрные ящики).
- Интеграция с популярными библиотеками (XGBoost, LightGBM, CatBoost, scikit-learn, TensorFlow, PyTorch).

SHAP прошёл путь от академической разработки до индустриального стандарта. Сегодня его используют в медицине, финансах, промышленности, HR и многих других областях для обеспечения прозрачности и доверия к моделям машинного обучения.

Прилагаемый Jupyter Notebook содержит практические примеры с кодом на Python, демонстрирующие все описанные в пособии визуализации и техники. Рекомендуется запустить его и поэкспериментировать с параметрами, чтобы закрепить понимание.

10. Список литературы

- Lundberg, S. M., & Lee, S. I. (2017). "A Unified Approach to Interpreting Model Predictions." *Advances in Neural Information Processing Systems (NeurIPS)*, 30.
- Shapley, L. S. (1953). "A Value for n-Person Games." *Contributions to the Theory of Games*, 2(28), 307–317.
- Lundberg, S. M., Erion, G., Chen, H., et al. (2020). "From Local Explanations to Global Understanding with Explainable AI for Trees." *Nature Machine Intelligence*, 2(1), 56–67.
- Molnar, C. (2022). "Interpretable Machine Learning: A Guide for Making Black Box Models Explainable." <https://christophm.github.io/interpretable-ml-book/>
- Документация библиотеки SHAP: <https://shap.readthedocs.io/>
- Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). ""Why Should I Trust You?": Explaining the Predictions of Any Classifier." *Proceedings of the 22nd ACM SIGKDD*.
- Friedman, J. H. (2001). "Greedy Function Approximation: A Gradient Boosting Machine." *Annals of Statistics*, 29(5), 1189–1232.
- Strumbelj, E., & Kononenko, I. (2014). "Explaining Prediction Models and Individual Predictions with Feature Contributions." *Knowledge and Information Systems*, 41(3), 647–665.

Приложение: Jupyter Notebook с примерами

Вместе с этим пособием поставляется файл `shap_analysis.ipynb` — Jupyter Notebook, содержащий практические примеры использования SHAP для задач регрессии и классификации.

Содержание ноутбука:

- Импорт библиотек и настройка окружения
- Задача регрессии (California Housing): загрузка данных, обучение Random Forest, SHAP TreeExplainer
- Все основные визуализации: Summary Plot, Bar Plot, Waterfall Plot, Dependence Plot, Force Plot, Decision Plot
- Сравнение Random Forest и XGBoost через SHAP
- Задача классификации (Breast Cancer Wisconsin): обучение классификатора и SHAP-анализ
- Демонстрация KernelExplainer для произвольной модели
- Анализ взаимодействий признаков (SHAP Interaction Values)

Для запуска ноутбука потребуются библиотеки:

```
pip install shap scikit-learn xgboost pandas numpy matplotlib
```

Запустите Jupyter Notebook или JupyterLab в директории с файлом `shap_analysis.ipynb` и выполняйте ячейки последовательно. Экспериментируйте с параметрами моделей, меняйте наборы признаков, пробуйте разные эксплейнеры.